Exploring Stealthy Spatiotemporal Attacks in Mixed Reality

Anonymous Author(s)

ABSTRACT

Mixed Reality (MR) is rapidly becoming an essential technology for critical applications such as physical therapy and surgery, in addition to its early use for leisure activities. This transition necessitates a focused look at the security aspects of MR devices and applications. Prior work on MR security has primarily focused on generic security aspects such as secure authentication and vulnerability analysis. However, MR devices are multi-modal and spatiotemporal in nature, which exposes them to attacks on sensor modalities across spatiotemporal axes. Prior work has demonstrated attacks on individual sensing streams, but modern, state-of-the-art sensor fusion-based tracking algorithms can easily mitigate such attacks.

In this paper, we introduce multi-modal spatiotemporal attacks that concurrently attack multiple sensing streams across spatiotemporal axes to yield *fast*, *stealthy*, and *precise* outcomes. To the best of our knowledge, our work is the first to propose, design, and evaluate concurrent multi-modal spatiotemporal attacks. In doing so, we solve key challenges related to deciding on what attack mechanisms to use, when to launch the attacks, and how to configure attacks. Using the tracking and navigation use case of a user wearing an MR headset, we demonstrate that our attacks enable stealthy and precise control over the user's trajectory in presence of state-of-the-art sensor fusion-based tracking algorithms and system checks.

1 INTRODUCTION

The immersiveness of Mixed Reality (MR) applications relies on multi-modal sensing of user activities and surrounding environment via commodity sensing devices such as Head Mounted Displays (HMD), hand-held controllers, and other devices [2, 15]. MR systems center around tracking humans and enhancing their experience by designing human-in-the-loop systems. A key challenge for human-in-the-loop systems is the exploitation of security vulnerabilities that impact human safety [39]. Due to the prevalence of sensors, reduced field of view, and integration of critical applications, MR devices can be a lucrative target for malicious activities with real-world consequences to human safety [19]. Recent security trends in MR mostly focus on security and privacy issues around authentication [32], access control [26], and digital biomarkers [29]. However, we argue that there are additional attack surfaces, more prevalent and stealthy, that pose physical harm to MR users.

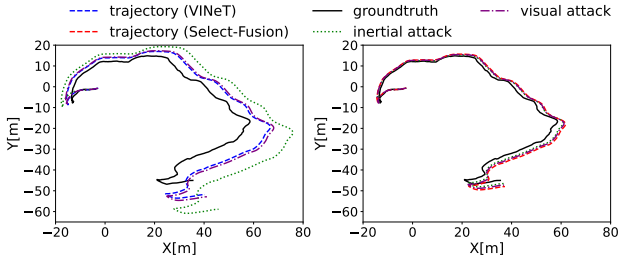
Emerging technologies—in MR, autonomous vehicles and robotics—depend on the macro- and micro-level tracking of humans. These tracking algorithms leverage multi-dimensional spatiotemporal relationships among multi-modal sensing streams. For example, Visual-Inertial Odometry (VIO) tracking algorithms in MR combine data from visual cameras and inertial sensors. Recent studies have devised attacks on individual sensing modalities such as inertial [33] and visual [5] sensors. However, state-of-the-art deep learning-based sensor fusion techniques such as Select-Fusion [7] can successfully mitigate attacks on individual sensors. In sensor fusion-based tracking, if data from one sensing stream degrades, the tracking algorithm prioritizes others streams to ensure correct operation. In addition, such MR systems are highly interactive and have tight time-constraints for successful attacks. A visual lag or a glitch can forewarn a user of a potential manipulation.

In this paper, we present a new attack surface that is resilient to the deep learning-based sensor fusion tracking algorithms such as Select-Fusion [7]. Our attack surface introduces concurrent attacks on multiple sensing streams and coerce sensor fusion algorithms to adapt to the manipulated trajectory. Crafting these attacks is not trivial as they require precise control over multi-modal sensors across appropriate spatial and temporal axes. The goal of the attacker is to bypass the system and user checks, and diverge a user wearing an MR headset from its original trajectory in an intentional, precise yet stealthy manner.

We argue that frame-level spatiotemporal manipulations (such as compress, stretch, shuffle, misalign) across appropriate sensing modalities set the basis of our work, and we devise a set of attacks—such as slowdown, speedup, zero displacement, and path deviation to demonstrate our attack surface. However, launching *fast* and *stealthy* attacks to avoid detection, and *precise* attacks to yield desirable outcomes on MR devices present multiple challenges. First, what attack mechanisms should be used to launch *fast* attacks? Second, when to attack the sensing streams to ensure stealthiness? Third, how to control the attack mechanisms to launch precise attacks? While the simplicity of frame-level manipulations enables fast attacks, crafting attacks that achieve a long-term goal, such as deviating a user from her path, using instantaneous frame-level manipulations in a time-constrained environment is challenging.

The novelty of our work is not in devising new frame manipulations, it is rather in building policies on top of the existing manipulations to concurrently attack multiple sensor streams across spatiotemporal axes at the right time and for the exact duration. We devise an algorithm that determines when to launch an attack based on similarity between consecutive frames. To achieve *stealthiness*, our algorithm relies on similarity index based on inter-frame correlation or hashing to ensure the attack is launched only when its effect can be masked as a normal variation in data. It also keeps track of similarity indices across multiple modalities for *concurrent* launch. Finally, it parameterized the attack mechanisms for a configurable knob to *precisely* control the effect of attack.

For the purpose of this work, we focus on an MR platform, such as HoloLens 2 [1], that relies on inertial and visual sensing streams for tracking. While we focus on inertial and visual sensors, our work is applicable to any other sensors that have a temporal and spatial component to it. We define multiple attacks where each is a combination of an attack mechanism for both inertial and visual sensing streams, an algorithm that leverages the attack mechanism to launch attacks across spatial and temporal axes, and a set of configuration parameters for the attack mechanism and the algorithm. We demonstrate the efficacy of our proposed attacks for a tracking and navigation use case where a user wears an MR headset.



(a) VINeT (b) Select-Fusion

Figure 1: *State-of-the-art sensor fusion algorithm, Select-Fusion [7], is effective against attacks on a single sensing stream whereas simple tracking algorithm, VINet [8], fails.*

Our hypothesis is that our concurrent spatiotemporal attacks on multiple sensing streams yield *fast*, *stealthy*, and *precise* outcomes. In evaluating our hypothesis, we make the following contributions:

Introduce Multi-modal Spatiotemporal Attacks. We introduce the concept of multi-modal spatiotemporal attacks in MR systems. Prior work has proposed attacks on individual sensing streams, which can be easily mitigated by sensor fusion-based tracking algorithms. To the best of our knowledge, our work is the first to propose concurrent attacks on multiple sensing streams across spatiotemporal axes to circumvent state-of-the-art tracking algorithms. However, launching concurrent spatiotemporal attacks across multiple sensing streams is non-trivial. Our work devises a framework, based on similarity index, that allows launching concurrent attacks without explicit coordination between attack-enabling mechanisms.

Design *Fast, Stealthy,* and *Precise* Attacks. We first define a comprehensive threat model for our tracking and navigation use case. We then propose attacks that leverage simple frame manipulations as attack mechanisms to launch fast attacks. Our similarity index-based framework to launch concurrent attacks also informs the timing of our attacks to achieve stealthiness. Finally, we devise a Right Frame at the Right Sensor (RFRS) algorithm that uses empirically-determined configuration parameters to launch precise and highly controllable attacks. In devising each solution, we solve many research and practical challenges, outlined in §4.

Evaluate Fastness, Stealthiness, and Preciseness of Attacks. We extensively evaluate our proposed attacks to demonstrate their speed, stealthiness, and preciseness. Using the example of a user in an office building, we demonstrate precise control over the user's trajectory across all our attacks in a stealthy manner even in presence of state-of-the-art sensor fusion algorithms and system checks.

2 MOTIVATION AND PROBLEM STATEMENT

In this section, we use an illustrative example to motivate the need for launching multi-modal spatiotemporal attacks to enable *fast*, *stealthy*, and *precise* outcomes. We also outline the problem statement and define relevant key terminologies.

2.1 Motivating Example

Emerging applications across domains leverage multi-modal sensing to enable the tracking of humans in a given environment. Prior work has successfully demonstrated attacks on single sensors streams. For example, Trippel et al. [33] demonstrate that an attacker can precisely manipulate the data from an inertial sensor using acoustic signals. Vanhoef [37] and Valente et al. [36] show that an attacker can manipulate visual stream by compromising the network link between camera and the tracking device. We present an example that investigates the efficacy of such attacks against tracking algorithms, where the goal of the attacker is to deviate the user wearing an Mixed Reality (MR) headset from its path and increase the total distance traveled.

Figure 1 illustrates the working of attacks on inertial and visual sensing streams against two tracking algorithms: VINet [8] and Select-Fusion [7]. VINet is a simple visual-inertial odometry-based tracking algorithm while Select-Fusion is the state-of-the-art tracking algorithm that leverages sensor fusion to mitigate attacks on individual sensing streams. We observe that, for VINet, even an individual attack on either visual or inertial sensor can be successful. However, for Select-Fusion, the tracking algorithm is able to detect and mitigate the individual attacks, while a concurrent attack on both streams, proposed in this paper, successfully evades the correction mechanism. This is because the sensor fusion algorithm no longer has a normal reference sensing stream to detect and correct for the attacks. This insight motivates the design of concurrent attacks on multi-modal sensing streams across spatiotemporal axes.

2.2 Problem Formulation

Before outlining the problem statement, we define the key concepts of *Attack Surface*, *Attack Knobs*, and *Attack Vectors*.

- **Attack Surface.** An attack surface is the collection of *attack vectors* where an attacker can manipulate data in an environment. In this paper, the tracking of a user in an MR environment is the attack surface and output of the inertial and visual sensors, and its processing is prone to manipulation by the attacker.
- **Attack Vectors.** An attack vector is the path taken by an attacker to exploit the vulnerabilities in an *attack surface*. In this paper, the inertial and visual sensor modalities are the attack vectors as they have the requisite temporal and/or spatial features.
- **Attack Knobs**: An attack knob specifies the dimension of an *attack vector* along which the data can be manipulated by the attacker. In this paper, spatial and temporal axes of the inertial and visual *attack vectors* are our attack knobs.

We next outline the problem statement for our work, given the definitions and specific examples of the *attack surface*, *attack vectors*, and *attack knobs* used in this paper.

Problem Statement. Our problem statement is - "*how to devise attacks that provide precise control of user's trajectory to an attacker while successfully evading the state-of-the-art sensor fusion-based tracking algorithms and system checks in an MR environment?*" To solve this problem, our hypothesis is that *concurrent attacks that use appropriate attack knobs (spatial and temporal) on multiple attack vectors (visual and inertial) enable fast, stealthy, and precise outcomes.* We next outline key challenges in evaluating our hypothesis.

(1) What attack mechanism to use to launch ***fast*** attacks?
(2) When should the attack be launched to ensure ***stealthiness***?
(3) How to configure the attacks to yield ***precise*** outcomes?

Our proposed attacks (§4.3) rely on simple frame manipulations that are not time- or compute-intensive (§4.2) to enable ***fast*** attacks. We devise an algorithm (§4.5) to determine the time of the attack to ensure ***stealthiness***, not only against sensor fusion algorithm

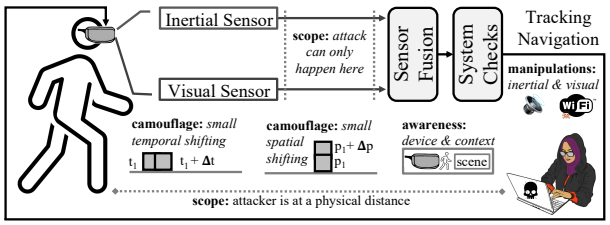


Figure 2: *Threat Model Overview.*

but also system checks. Finally, we conduct an experimental study to configure the attack to yield *precise* outcomes (§4.4).

3 THREAT MODEL

In this section, we present the threat model for the proposed multi-modal spatiotemporal attacks. We consider the scenario where an attacker is attempting to diverge the trajectory of a user wearing a Mixed Reality (MR) headset, as illustrated in Figure 2. We assume that the user is using a tracking and navigation application that guides her on which direction to take through either visual guidelines or an avatar walking alongside her. It also informs her about her estimated speed. The application continuously monitors and tracks her location using a sensor fusion-based tracking algorithm, and provides feedback or instructions to her through the headset display. We next describe the various aspects of our threat model and our scenario-specific assumptions in reference to Figure 2.

Attack Scope. We assume that the attacker can neither directly access the digitized sensor readings nor physically touch the sensors on the headset. She can only exploit vulnerabilities when the data is being generated on the sensor level and has not been moved to any of the data processing pipelines (e.g., tracking, sensor fusion). Also, she does not have physical access to the device or its firmware but is in close proximity to the headset.

Device & Scene Awareness. While the attacker cannot physically access the device, she has access to a similar device and can learn the behavior of sensor fusion algorithms, system verifiers, frame rate of sensors, architecture, and parameters. Also, since she is close proximity, she can visually see the user and the surrounding environment to launch activity-aware attacks [30, 40].

Sensor Data Manipulation. We assume different access mechanisms and manipulation schemes for our inertial and visual attack vectors. For inertial attack vector, prior work has shown that an attacker can manipulate the analog signals on the signal conditioning path, before the data is digitized or processed, by emitting acoustics signals [33]. In §4.2, we demonstrate how we devise the notion of frames for the inertial data and discuss how manipulations of analogue acoustic signals from prior work can be used to manipulate the inertial frames. Given this notion, the attacker can achieve inertial frame manipulations needed for our attacks that include stretching, compressing, and shifting sensor measurements.

For the visual attack vector, we assume that she has compromised the network link between the camera and the data processing pipeline of the headset, as demonstrated in prior work [37]. We also assume that she has the ability to buffer a significant amount of visual frames [36]. Given such capabilities, the attacker can easily achieve the visual frame manipulations including duplicating, dropping, and controlling the change between consecutive frames.

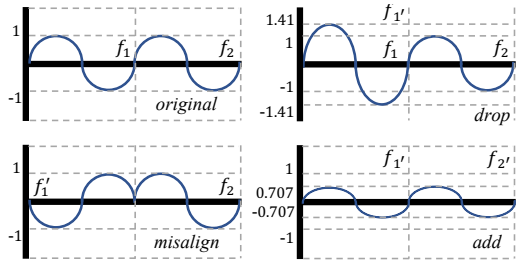


Figure 3: *Notion of frames for inertial sensors and how amplitude modulation attacks in WALNUT [33] allow adding, removing, and misaligning frames.*

Camouflage. The attacker will essentially use two types of alteration in the frames: spatial and temporal. All the alteration when combined across different sensors can look like small errors which can be ignored by the tracking algorithm. Like spatial alteration can be perceived as misalignment that can occur due to axis misalignment and the incorrect sensor calibration [21]. The time shift between windows of input images and windows of inertial measurements can be perceived as a relative drift in clocks between independent sensor subsystems [22].

User Behavior Assumptions. We assume that the user is either wearing an opaque MR headset, where she assesses the physical world through a video stream, or a see-through MR headset, where she can see the physical world. In the former case, post attack, she will only see the manipulated visual stream and cannot tell if the stream deviates from the real world, making the attack stealthy. In the latter case, she will be able to see the world directly and only the tracking system of the headset is manipulated. Here, an attacker can deviate her from its path by manipulating the guidelines that are being delivered either using visual instructions or an avatar walking alongside her in the tracking and navigation case.

Attacks Concurrency. Given our attacker is in close proximity to the user, she can use her smartphones to launch the attacks. Inertial attacks use acoustic signals from speakers and visual attacks compromise the network link. Given the computing power of modern smartphones and embedded devices, it is possible to launch both attacks concurrently. Furthermore, prior work [33, 36] has already demonstrated that both attacks are individually successful, and hence not in scope of our work.

4 DESIGNING SPATIOTEMPORAL ATTACKS

In this section, we present our approach to designing *fast*, *stealthy*, and *precise* attacks in a Mixed Reality (MR) environment.

4.1 Attack Setup

Before we describe the mechanisms that enable our multi-modal spatiotemporal attacks, we will discuss how the attacks on each individual stream are launched for a setup described in §3 and illustrated in Figure 2. Our work uses the notion of *frame*, which corresponds to a single image from the visual stream and a single measurement from the inertial sensor. As we discuss in the next section, we use frame-level manipulations such as adding and dropping frames as our attack-enabling mechanisms.

For the visual stream, if an attacker has compromised the network link [37] and has the capacity to buffer a finite number of packets [36], she can drop frames or duplicate frames in the stream.

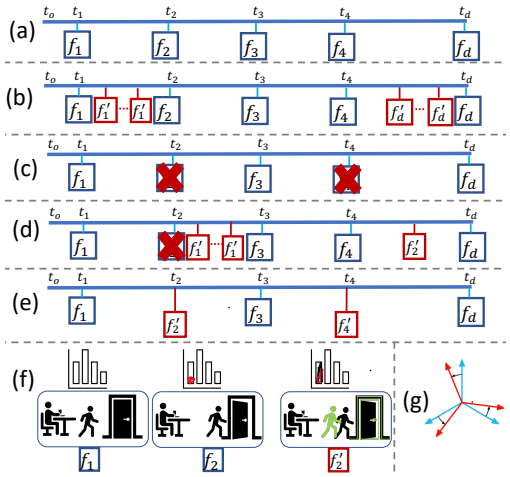


Figure 4: ***Various frame manipulation techniques. (a) original, (b) stretch, (c) compress, (d) shuffle, (e) visual data misalign, (f) histogram shift, and (g) inertial data misalign.***

The camera frame rates for modern MR headsets show significant variations. For example, HoloLens 2's PhotoVideo (PV) camera fluctuates in the range of 5-30 frames per second [1]. Therefore, slow addition or dropping of the frames by the attacker will be interpreted as normal fluctuations in the frame rate.

For the inertial stream, the notion of frames and frame-level manipulation is non-trivial. We term a single measurement from the inertial sensor as a frame, which essentially corresponds to the analog data collected from the inertial sensor during one sampling period. Figure 3 (top-left) shows the output of an hypothetical accelerometer along one axis for two sampling periods. To drop a single frame, we double the acceleration which gives an impression of users walking longer than actual. This can be achieved by multiplying the magnitude along each axis by $\sqrt{2}$. Similarly, we can add a single frame by dividing the magnitude along each axis by $\sqrt{2}$ (bottom-right), as it slows down the sensor reading indicating the user is not moving. Similarly, we achieve a complete misalignment required for the spatial knob by multiplying the signal with -1 (bottom-left), that represents a 180^o shift in user orientation. Trippel et al. [33] show that the amplitude modulation can be used to enable a highly precise control over sensor readings to the extent that they spelled out the word WALNUT using sensor output. The control mechanism required for our approach can be launched in practice. Since adding or dropping a frame in both attack vectors is not compute-intensive, both attacks can be concurrently launched.

4.2 Mechanisms for Spatiotemporal Attacks

As outlined in §2, a key challenge for the attacker is to launch *fast* attacks, primarily motivated by the fast sampling rate of sensors. However, fast attacks also facilitate stealthiness by allowing attackers to make slow, small changes rather than sudden and large manipulations. As a result, we use frame-level manipulation techniques as the enabling mechanism for our attacks. The frame manipulations used in our attacks are not novel. Our contribution lies in leveraging these simple manipulations to launch a wide-range of attacks described in § 4.3. We next describe the different types of manipulations that we use in our attacks.

Stretch. In this manipulation, the attacker temporally *stretches* the sensing stream by adding more frames between two consecutive frames, as demonstrated in prior work [31]. For the visual stream, the attacker will duplicate the images sent to the sensor fusion algorithm's visual processing pipeline. For the inertial stream, the attacker reduces the amplitude of the inertial sensing signal to create the "add frame" effect. Reducing the amplitude to 0 is equivalent to adding infinite duplicate images for the visual stream. Figure 4(a) shows the non-attacked sequence of frames. While Figure 4(b) shows the sequence of frames after the attacker has induced two copies of malicious frame $f_1^{'}$ between frames f_1 and f_2.

Compress. In this approach the attacker manipulates frames temporally. She *compresses* the sensing streams by dropping frames, as demonstrated in prior work [16]. For the visual stream, the attacker will not forward some of the packets, which is akin to a camera operating at a lower frame rate. However, the scene in the images forwarded would be changing fast. For the inertial stream, the attacker can increase the magnitude of the measurements to give an impression of dropped frames. Increasing the magnitude to the maximum value is functionally equivalent to generating images at the highest frame rate for the visual stream. As shown in Figure 4(c), as frames drop, the sequence of frames seen by the tracking algorithm is changed. For example, if the attacker drops f_2 and f_3, f_4 will become new $f_2^{'}$ and so on.

Shuffle. This approach manipulates the frames both temporally and spatially. In a temporal manipulation, the attacker can switch between adding and dropping frames, as shown in Figure 4(d). For example, attacker can drop f_2 at its original time and buffer it to use it in place of f_6. It may also add some forgery in the frames to corroborate with the current tracking scenario.

Misalign. In this spatial manipulation, the attacker changes the spatial locality of the visual and inertial frames. Tracking algorithm looks at the f_i to f_{i+m} frames to determine the position and orientation of the user, where m is the window size of history for the tracking algorithm [28]. For the visual frame, it can be achieved by adding padding to the image on the desired side and changing the location of objects in the environment. For the inertial frame, it can be achieved by inverting the inertial sensor measurement. Figure 4(e) depicts a visual misalignment while Figure 4(g) shows the misalignment for the inertial sensor.

Histogram Shift. Histogram of an image shows the density distribution of pixel intensities and provides information about the objects in a given image. In this approach, the attacker manipulates an image such that the difference in histogram between adjacent frames does not exceed a threshold, whose value is determined empirically. One method to launch this attack is where the attacker takes a difference between the current frame i and the previous frame $i-1$. She then uses edge detection to identify the segments of the image that are different from the previous frame. Finally, she applies this "difference image" to the new image such that the histogram difference does not exceed the aforementioned threshold value. The goal of finding and respecting the right threshold is to avoid making drastic changes and remain stealthy. This allows the attacker to manipulate the spatial map of the surroundings leading to inaccurate positioning and tracking, as shown in Figure 4(f).

A similar attack for the inertial sensor can be achieved using the misalignment attack mechanism.

4.3 Multi-modal Spatiotemporal Attacks

In this section, we present multiple concurrent spatiotemporal attacks that use simple attack mechanisms discussed in § 4.2. We designed four attacks to demonstrate the validity of our attack surface. The goal of the attacker is to diverge the user's "reality" by leveraging concurrent manipulation of at least two attack vectors. The choice of manipulations depends on the attack requirements. We next describe the proposed attacks (summarized in Table 1).

Slowdown Attack. In this attack, an attacker uses the *stretch* mechanism to add frames in the sequence to give the tracking application an impression of the user slowing down. The attacker uses temporal knobs to concurrently manipulate inertial and visual vectors. To do so at time i, she can decide to hold the last frame that arrived at time $i-1$. Alternatively, as she has a buffer capacity, she can replay a sequence of frames from the history. In our design, we use the holding approach as it is simpler and faster to implement in practice. The end effect is that the tracking application perceives the user has slowed down. Note that the same attacking approach is concurrently launched against both the inertial and visual streams.

This result makes it an ideal attack when a user is using an MR-based running or jogging application that actively tracks and guides the user, similar to the prior work for campus navigation [23]. For instance, the attacker can use a slowdown effect on a user trying to jog at a fixed pace. This slowdown effect will deceive the application to estimating user jogging slowly, and respond by asking the user to run faster leading to harmful consequences [17].

Speedup Attack. The goal of this attack is opposite of the slowdown attack. The attacker will use the *compress* mechanism to concurrently drop frames from both the inertial and visual sensing streams. It will give the application an impression of the user speeding up. The attacker uses the same temporal knobs to manipulate the inertial and visual attack vectors exposed by our attack surface. This attack is also suited for the same application discussed in the previous attack. In our example scenario, the speedup illusion caused by the attack may result in the application asking the user to jog at a slower pace, leading to an ineffective jogging session.

Zero Displacement Attack. This attack takes the user from the *source* to the *destination*, but through a different path, resulting in a zero displacement for the overall trajectory. This attack uses a combination of *stretch*, *compress*, *shuffle*, and *histogram shift* mechanisms to ensure that the distance traveled by the user is also the same as the original trajectory. Unlike the previous two attacks that just used temporal knobs, this attack also exercises the spatial knob using histogram shift to mask the differences in the frames when they are shuffled. Just like previous attacks, any attack mechanism is concurrently applied to both the inertial and visual sensing streams. This attack is useful when the attacker wants the user to go through a specific point outside the trajectory or she wants the user to navigate from *source* to the *destination* at an uneven pace.

Path Deviation Attack. The attacker concurrently employs *misalign* and *histogram shift* attack mechanisms for the inertial attack vector and the visual attack vector, respectively. The tracking application is fed the wrong pose information about the user, which results in an incorrect trajectory estimation from the application. As a result, the user trajectory will deviate from the original and she will lose the destination. It is easy to envision how this can be very dangerous for simple navigation or jogging/walking use cases.

4.4 Key Challenges

Our proposed attacks employ *simple* attack mechanisms using various *attack knobs* to attack multiple *vectors* for a given goal. However, launching these attacks in a *fast*, *stealthy*, and *precise* manner poses significant challenges that we next outline and solve.

Challenge 1: How and when to launch concurrent attacks across vectors? Previous sections outlined "how" to attack a given vector, but did not discuss "when" should these attacks be launched and how the attacks across vectors are launched concurrently. Our challenge was to find a parameter or a metric that can act as a reference point between the inertial and visual attack vectors, as well as guide on when the attack be launched.

Prior work observes that ***similarity*** between frames is an important metric in determining user pose and semantic similarity for tracking and visual inertial odometry [43]. Given this information, we take the **similarity index** as the basic metric for deciding when to attack a given vector. Similarity index is a ratio of semantic overlap between adjacent frames f_i and f_{i+1}. A value of 0 means that there is no overlap between two frames. A value of 1 means the two frames are the same. To find the similarity index, we use hashing of a frame as the underlying metric and quantify the similarity between two hash keys as our similarity index. We can use other underlying metrics for the similarity index as long as they quantify the geometrical information for the given attack vector. We use hashing as it is robust to minor changes in frames and is used in detecting errors and flagging inappropriate data frames [14]. For the visual frames, we use perceptual hashing to quantify similarity index [27], which examines the contents of an image and constructs a hash value that uniquely identifies an input image. Given the hash values for frames, we compute the similarity index as the hamming distance between the hashes [18]. For the inertial frames, we use locality sensitive hashing and scale-insensitive cosine distance, defined as an angle between two vectors, as the similarity index [41].

To launch concurrent attacks, we set conditions on the values of similarity index for both vectors. The attacks are launched when those conditions are met. The conditions on similarity index are defined in terms of percentile on the similarity index values over the history window. For example, a 30%ile condition means that the value of the similarity index should be within 30%ile and 70%ile range for the attack to be launched. Since both sensors are tracking the same scene, their similarity index values are highly correlated. As we set the same conditions on both attack vectors, it allow us to launch concurrent attacks and are informed by our need for stealthiness, which we discuss in the next research challenge.

Challenge 2: How to attack multiple vectors in a stealthy manner? As discussed in the previous challenge, the similarity index can take a value between 0 and 1 depending on how similar are the adjacent frames. If the similarity index does not change over some time period, it means that the scene is not changing and vice versa. We propose that, for an attack to be stealthy, it should be launched when the similarity index has achieved a steady-state. The hypothesis is that, during the steady-state, the attack will be stealthy

Attack	Attack Mechanism	Attack Vector	Attack Knob	End Effect
Slowdown	Stretch	Camera	Temporal	Applications perceives user slowdown and prompts user to walk/jog faster.
		IMU	Temporal	
Speedup	Compress	Camera	Temporal	Applications perceives user speedup and prompts user to walk/jog slower.
		IMU	Temporal	
Zero Displacement	Stretch, Compress, Shuffle, Histogram Shuffle	Camera	Temporal, Spatial	User reaches the destination through a different path.
		IMU	Temporal	
Path Deviation	Histogram Shift, Misalign	Camera	Spatial	User is unable to reach the destination.
		IMU	Spatial	

Table 1: *Summary of proposed multi-modal spatiotemporal attacks in MR tracking and navigation.*

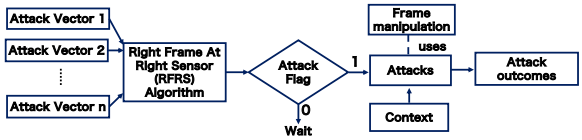


Figure 5: *Our approach to launching concurrent fast, stealthy, and precise spatiotemporal attacks.*

as the small frame manipulations that the attacker will make will be interpreted as the variations in similarity index that are already part of the sensing stream. The attack should not be launched during the transient state as it may result in drastic changes in the scene that can get detected. Furthermore, the launch of attack during steady-state allows the concurrency between attack vectors, e.g., attacker may launch attacks when both vectors are in steady-state. This choice also provides the attacker an ability to launch attacks only in a certain context (steady-state) when the scene is not changing. Finally, an attack in the steady state will allow better control, as we discuss in the next research challenge.

Challenge 3: How to enable a fine-grained and precise control over an attack? In previous challenges, we identified the metric that we use to launch the attack, how to concurrently attack across vectors, and how to make the attacks stealthy. While this information is enough to launch an attack, it does not explore how an attacker can achieve a fine-grained and precise control over the results. To provide precise control over the outcome, we parameterize the attacks to enable the attack launch only during the steady-state, a stealthy continuation, and a graceful end. These parameters configure our Right Frame at the Right Sensor (RFRS) algorithm. We next discuss the parameters and our algorithm.

4.5 Right Frame at Right Sensor Algorithm

Our attack approach (illustrated in Figure 5) takes in the sensor streams of different vectors, uses the Right Frame at the Right Sensor (RFRS) algorithm to decide when to attack, picks an attack based on the attacker's goal, and leverages our mechanisms to launch the attack. Algorithm 1, Right Frame at Right Sensor (RFRS) is our solution to enabling and configuring precise attacks. We next define the key terminologies used in the algorithm.

History Window (*history_window*): Number of frames that an attacker will take in to account for making a steady-state decision.

Steady-state Window (*steadystate_window*): It is the number of frames that need to be within a given range to achieve steady-state. A bigger window yields less opportunities to launch an attack. A smaller window launches more attacks and risks getting caught.

Steady-state Bounds (*upper_bound*, *lower_bound*): An attack vector is in a steady-state when its similarity index values remain in a given range for the duration of a steady-state window. The values of upper and lower bounds for the range are Xth percentile (Xp) and 100-Xp of similarity index in the history window, respectively.

Attack Length (*attack_length*): Number of consecutive frames that an attacker can attack in a given segment.

Now that we have described all the key terminologies, we next describe how the RFRS algorithm works.

Right Frame at the Right Sensor (RFRS) Algorithm. Algorithm 1 shows an automated mechanism for picking the frames to attack while staying stealthy. It takes a stream of *frames* and other attack parameter (configured by attacker) as an input and outputs the $attack\_flag$. If the $attack\_flag$ is 0, it is not safe to initiate or continue an attack. If the $attack\_flag$ is 1, the attacker can initiate or continue attack. To ensure concurrency, we only launch attack when the $attack\_flag$ is 1 for all the attack vectors.

The inner mechanism of the RFRS algorithm is purposefully simple. It buffers the incoming frames in a First In First Out (FIFO) buffer of size *history_window*. Once the history buffer is full, the algorithm computes the similarity index for all the frames in the buffer. After that, it finds the upper- and lower-bounds for the steady-state based on the attacker-configured parameters. If the similarity index lies in the specified range, a counter for the steady-state is updated. A steady-state is reached if the scene is consistent and the similarity index remains in the specified range for a period of *steadystate_window*. Once the steady-state is achieved, the RFRS algorithm will signal the attacker to initiate the attack. The attack will continue until the similarity index value goes outside the specified range or the number of frames attacked reach the *attack_length*. In this case, all the counters are reset and the algorithm starts searching for the next steady-state.

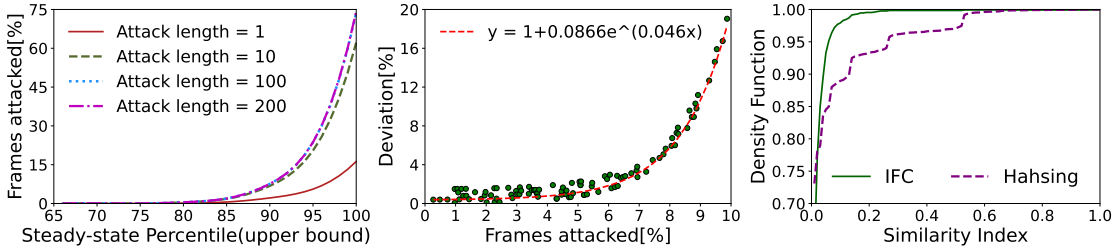
We demonstrate in the evaluation that by controlling the value of the RFRS input parameters, an attacker is able to precisely control the effect of an attack in a fine-grained, and stealthy manner.

5 EVALUATION

In this section, we evaluate the efficacy, stealthiness, and preciseness of our proposed concurrent multi-modal spatiotemporal attacks.

5.1 Experimental Setup

In our evaluation, we consider indoor positioning and tracking as our use case under two scenarios. In the first scenario, a user is trying to navigate an unfamiliar building to reach a specific destination without human assistance. MR localization and tracking application guides users by showing the current location and the path to follow. It helps reduce the cognitive effort required in navigating the unfamiliar and/or complex spaces and boost engagement through multisensory interaction. In the second scenario, the user



(a) Frames Attacked vs Bounds **(b) Deviation vs Frames Attacked** **(c) Inter Frame Correlation vs Hashing**

Figure 6: *Configurability and preciseness of Right Frame at the Right Sensor algorithm.*

Algorithm 1: Right Frame at the Right Sensor algorithm: Configuring the Start and the End of an Attack.

Input: $frame$,$history\_window$, $upper\_bound$, $lower\_bound$, $steadystate\_window$, $attack\_length$
Output: attack flag
attack flag ← 0
$history$ ← FIFO buffer of size $history\_window$
while *frames are coming* **do**
  history.append(frame)
  if *history buffer is full* **then**
    sim_index ← similarity index of adjacent frames
    low ← $percentile(history, lower\_bound)$
    $high$ ← $percentile(history, upper\_bound)$
    if sim_index *in range*$(low, high)$ **then**
      $steadystate\_count++$
    else
      $steadystate\_count$ ← 0; attack flag ← 0
    if $(steadystate\_count \geq steadystate\_window)$ *and* $(attack\_count \leq attack\_length)$ **then**
      $attack\_count++$; attack flag ← 1
  else
    attack flag ← 0

is running/jogging on an indoor track inside a gymnasium building. MR application guides the runner to have an effective run by maintaining a desired and safe jogging/running pace.

Data. To collect the real-world data, authors of the paper mimicked users under two scenarios by navigating office buildings and tracks on campus. Tracking data and the ground truth is collected using Hololens 2 operating in research mode [1, 35]. Visible Light Cameras (VLC) generated data at a frame rate of 5-30 frames per second (fps) and raw inertial sensor data was collected at 12-20 fps, both with timestamps [35]. We matched the timestamps of closest inertial data to the VLC data frame for sensor fusion. Overall, we collected 54032 samples (52 minutes with mean 17 fps) of data.

Tracking. Tracking algorithms are the key component of any positioning and tracking application. Our attacks feed malicious data to the tracking algorithm to mislead its estimate of the user's actions. In our evaluation, we use the state-of-the-art sensor fusion-based tracking algorithm Select-Fusion [7] that is highly effective against attacks on single sensors, as shown in §2.1. Our tracking module takes raw images and inertial frames to generate corresponding pose transformations. We train our model on our 54032 data samples with 75% - 25% train and test split. We base our implementation architecture on Select-Fusion and train in the stochastic fusion mode. We use PyTorch for implementation and trained on a NVIDIA GeForce RTX 2070 GPU with batch size of 8 using the Adam optimizer, with a learning rate $lr = 1e^{-4}$. Training and testing mean squared errors was 0.0095 and 0.017 for Select-Fusion, respectively.

Metrics. To evaluate how the overall system perceives the effect of our attacks over time, we use the Cumulative Density Function (CDF) of the similarity index over the attack sequence and compare it with the original sequence. CDF is commonly used to detect the attacks on the system [25] because typical frame distributions and relationship with subsequent frames do not change drastically in a short period of time. Our assertion is that if the overall frame distribution has not changed, as quantified by a CDF, the system is not perceived to be under attack by any system checks.

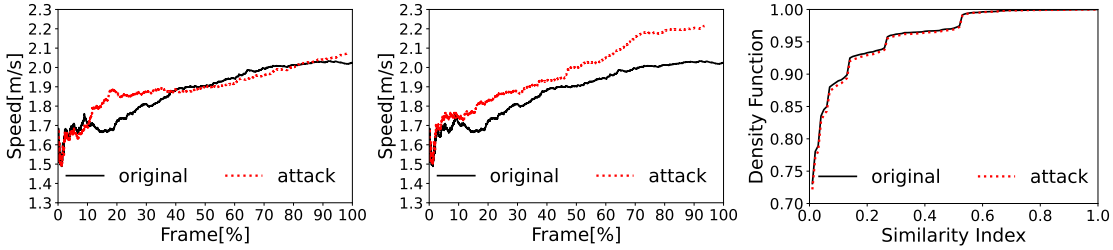
To quantify the similarity between different distributions, we compute Kolmogorov–Smirnov test [24]. For stealthiness, the attacker needs to keep this as small as possible as it represents how close are the distributions for the attacked and original sequence. We quantify the difference between arithmetic means of both sequences to find the variation among similarity indices. Higher the mean difference, more the difference between both sequences.

5.2 Configurability and Preciseness of Attacks

Our RFRS algorithm (Algorithm 1) offers an attacker multiple configurable parameters to launch precise attacks. These parameters determine what percentage of frames are attacked and how many frames are attacked in one attack sequence. To configure these parameters, she needs to know the relationship between the parameters and the ultimate deviation achieved in the user's trajectory. To quantify this relationship, we perform a set of experiments that vary the parameters and record their effect on trajectory.

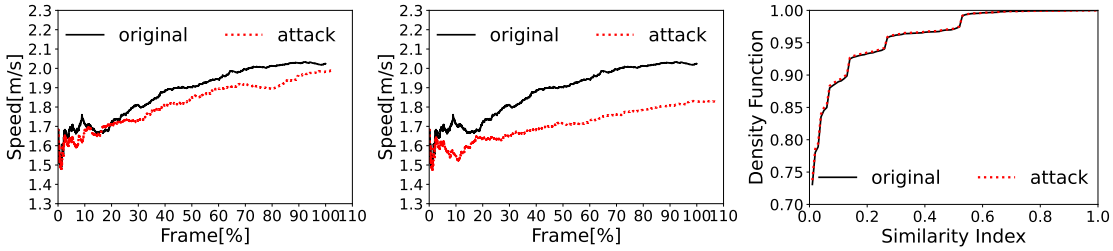
Figure 6a quantifies the relationship between steady-state bounds and the steady-state length parameters. To reiterate, steady-state bounds define the acceptable range for the similarity index values. Steady-state length controls how long the similarity index needs to be in range for a sequence to be labeled in steady-state We can see that if the bounds are set too low, there are not enough sequences of frames that can be attacked. For example, almost no frames can be attacked if the lower-bound and upper-bound are set to 20% and 80%, respectively. The number increases significantly at 90% upper-bound value and encompasses almost all the frames if the bound is set to 100%. The different curves show the effect of the steady-state length on the frames attacked. The total frames attacked increases as the attack length increases from 1 to 10, but there is only a marginal increase beyond that.

Figure 6a shows how many frames are attacked, but does not inform the attacker on how much deviation will be achieved if a particular number of frames are attacked. To do that analysis, we varied the number of frames attacked (by carefully configuring the attack parameters) and measured the deviation achieved under a wide-range user movement scenario. Figure 6b shows the actual data points and the fitted exponential curve to the data. We see that the deviation increases exponentially as the number of frames attacked increases. However, the exponential relationships dictates that the attacker would need to be really careful in devising the



(a) Speedup (attack length 1) **(b) Speedup (attack length 10)** **(c) Similarity Index (attack length 10)**

Figure 7: *Speedup Attack. The magnitude of Speedup depends on attack length (small at 1 (a) vs large at 10 (b)). (c) shows that the distribution of hashing function that does not change before and after the attack demonstrates stealthiness.*



(a) SlowDown (attack length 1) **(b) SlowDown (attack length 10)** **(c) Similarity Index (attack length 10)**

Figure 8: *SlowDown Attack. The magnitude of SlowDown depends on attack length (small at 1 (a) vs large at 10 (b)). (c) shows that the distribution of hashing function which does not change before and after the attack, demonstrating stealthiness.*

attacks as marginal change in deviation can be very large. Furthermore, as our threat model assumes that the attacker has access to a similar headset, she can determine this relationship offline. The relationship stays the same if the environment stays the same. However, the relationship may change if the environment changes and the attacker would either determine the new relationship empirically or get this equation from a third-party attacker.

Our evaluations point to a trade-off between the risk and reward for the attacker. To achieve a large deviation, she has to attack a larger %age of the frames, which may expose the attack. A small attack ensures stealthiness but may not achieve the desired deviation. This is further complicated by different magnitudes of the outcome for the same amount of change in the configuration parameters. As a result, the attacker should remain in a range that offers a more steady relationship between the attack parameters and the deviation. In our use case, this range lies between 85%-95% steady-state bound and the attack length of 1, where she attacks only 10% of the frames and achieves a deviation in the range of 2-20%.

We also analyze additional similarity metrics for their ability to capture the nuances in our scenario. We compared our chosen metric hashing with Inter Frame Correlation (IFC). IFC measures the correlation among consecutive frames. Figure 6c shows the CDF of hashing and IFC for a single sequences in our use case. IFC is unable to detect the different nuances of frames similarity index, but hashing paints a nuanced picture of our use case.

5.3 Efficacy and Stealthiness of Attacks

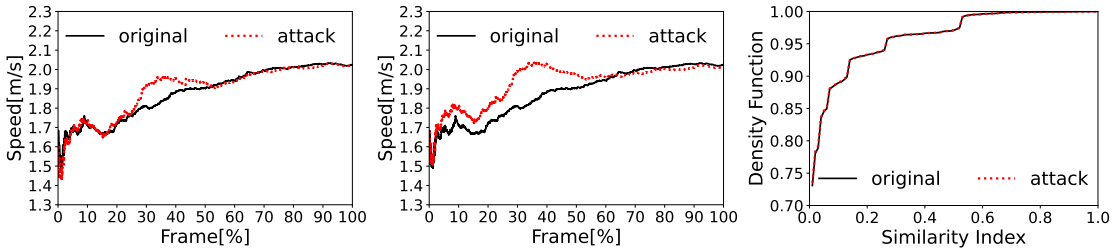
In this section, we evaluate the efficacy, preciseness of control, and stealthiness of our proposed attacks.

Speedup Attack. In a *speedup* attack, the attacker wants to give the tracking application an impression that the user is walking at a faster speed than her actual speed. As the tracking application is trying to maintain a constant pace for the user, it will instruct the user to reduce her speed, leading to an eventual slow down of the user. Figure 7 shows the effect of *speedup* attack on cumulative average speedup and similarity index distribution. To reiterate, an *speedup* attack is achieved by dropping frames causing the scene to change more rapidly. The tracking algorithm does not perceive this as a gap in time, but instead as the user moving faster. Figure 7a shows that the end gain in speed is very small after the attack as only a small number of frames were dropped. As the length of the attack is increased to 10, the increase in the perceived speed is significant as larger number of frames are dropped, as shown in Figure 7b. It is worth noting that our attacks are successful at both the small as well as a large attack lengths. This is because the tracking algorithm is unable to correct for the deviations at both scales due to the concurrent and multi-modal nature of the attacks.

Figure 7c demonstrates the stealthiness of *speedup* attack. While the attacker is able to increase the speed by around $0.3m/s$, it does not significantly change the overall distribution of similarity index. A sanity check mechanism that uses change in similarity index across frames would not be able to detect the attack. As mentioned above, the final effect of the *speedup* attack depends on the application characteristics. In the alternative scenario, the user trajectory will change if the application is trying to guide the user on a specific path. This will happen as the sequence of frames seen by our fusion algorithm, which relies on a time-series of data, changes post-attack.

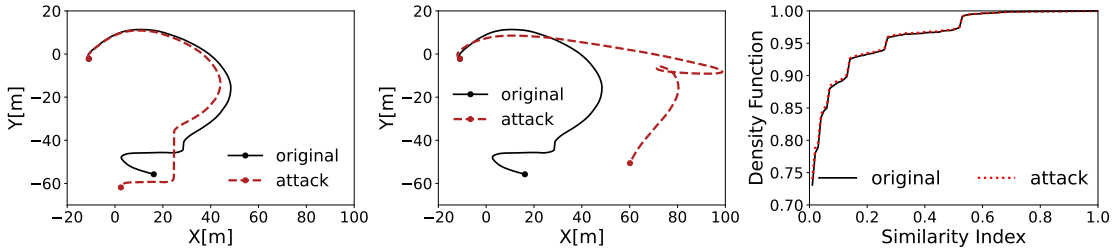
Slowdown Attack. In this attack, we consider the same example of a user with an objective of walking at a desired fixed speed. The expected result for a *slowdown* attack is that the application will perceive the user as walking slowly and generate a feedback for her to walk faster, leading to an actual speedup effect.

Figure 8 shows the effect of *slowdown* attack on the cumulative speed of the user and similarity index distribution. To reiterate, a *slowdown* attack is achieved by holding the frames causing the



(a) Zero Displacement (attack length 1) (b) Zero Disp. (attack length 10) (c) Similarity Index (attack length 10)

Figure 9: ***Zero Displacement Attack. The magnitude of deviation depends on attack length (small at 1 (a) vs large at 10 (b)). (c) shows that the distribution of hashing function which does not change before and after the attack, demonstrating stealthiness.***



(a) Trajectory (attack length 1) (b) Trajectory (attack length 10) (c) Path Deviation attack

Figure 10: ***Path Deviation Attack. The trajectory is shown with attack length of 1 (a) vs attack length 10 (b)). (c) shows that the distribution of hashing function as a similarity index does not change before and after the attack, demonstrating stealthiness.***

scene to change slowly, giving the application a perception of the user moving slower. The observed results are very similar to the previous attacks. At small attack lengths, slowdown is small and only a few attacks are added (Figure 8a). At larger attack lengths, slowdown is large and large number of frames are added (Figure 8b)

Figure 8c demonstrates the stealthiness of the launched attack. Similar to the *speedup* attack, the attacker achieves a speed slowdown of around $0.3m/s$, without significantly changing the distribution of the similarity index. While an instruction to walk faster defeats the purpose and deviates users from the trajectory, it can also have adverse physical consequences for people with underlying health conditions and bystanders in the situation.

Zero Displacement Attack. In a *zero displacement* attack, the attacker wants to give the application an impression that the user is periodically changing its speed, walking faster at times while slowing down at others. Since this attack is launched using a combination of *speedup* and *slowdown* attacks, the end effect depends on when each of the attacks is launched and if one type of the attack dominates the other. If both attacks are launched alternatively for the same duration, the net effect would be zero. The user would have just followed a different path to the original trajectory.

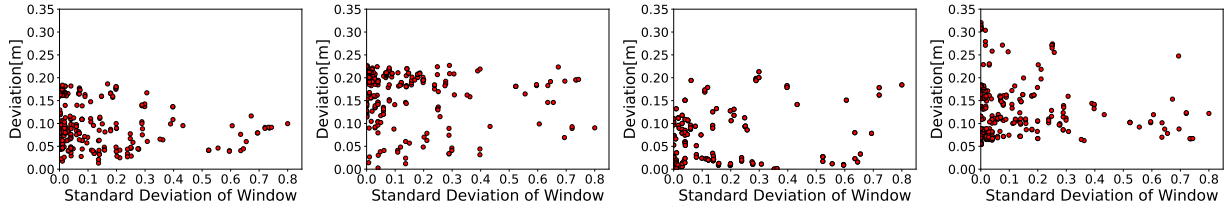
Figure 9 presents the results for zero displacement attack. Figure 9a and 9b show that speed increases slowly at the start, followed by a big divergence, and finally reduces towards the end. In practical scenarios, the attacker would not know the speed of the user in advance, so constant shuffling of frames can get the attacker to the final destination at the same time. However if this attack is handled only with a temporal knob the attacker might have diverted from the original destination even with shuffling of the frames. The attacker uses spatial knobs to bring the user back to the final destination. Similar to the previous two attacks, Figure 9c shows that there was no statistically significant difference between the similarity index of the original and attacked sequences.

Path Deviation Attack. Our final attack, the *path deviation* attack, demonstrates the effect of manipulation via spatial knobs on both the inertial and visual attack vectors. Here, the attacker wants to change the direction of the user and does not care about deviating her from final destination. Figure 10 shows the effect of *path deviation* attack on cumulative user speed and similarity index distribution. Figure 10a shows that the trajectory after the attack shifts by a small margin at small attack length. As expected, the change is significant for the larger attack length, as shown in Figure 10b. This is the long lasting effect of smaller changes throughout, the attacker did not change much but kept changing for longer duration and slowly the effect accumulated at the end. Overall the attacker achieved a 32 meters change in position and 1.5 radians change in the orientation. Finally, Figure 10c shows no significant difference between the similarity index of original and attacked sequences.

5.4 Effect of Steadiness on Attacks

As mentioned before, our proposed attacks are launched during the steady-state, which is defined as having *num_steady_state* frames within a certain bound. However, not all steady-states are the same and may have significant variations in the frames within the steady-state window. For example, if a scene is changing, frames would be similar and vice versa. We quantify this variation using standard deviation calculated over a given steady-state window and analyze the effect of this value against the deviation for that window.

Figure 11 shows that, across all proposed attacks, most of our attack sequences are launched in a window with lowest standard deviation between the similarity index. Some of the attacks are launched in highly variable windows as well. This can happen as a highly dynamic scene slowly transitions into a quiet scene. The steady-state can be achieved despite the high variations in the similarity indices, leading to an attack being launched.



(a) Speedup (b) Slowdown (c) Zero Displacement (d) Path Deviation

Figure 11: ***Effect of steadiness of a given window on the deviation achieved in that particular window. Low variation in a given window results in a more controllable and effective attack.***

Figure 11 also illustrates the impact of standard deviation in similarity indices on the deviation achieved. In most cases the low standard deviation, aka extremely steady-state, results in high deviation. This facilitates stealthiness and yields divergence intended by the attacker. This demonstrates a significant potential of our proposed attack surface and algorithm. MR is becoming common in critical applications like surgery [12] and rehabilitation [11]. In such applications, micro movements are tracked and the physical scene does not significantly change but movements are critical. Our attack surface works best in steady state and the attack will be effective in scenarios where the micro movements are essential, and attacking those can cause serious consequences. Even if the scene is not changing the deviation is still attainable in those scenarios.

5.5 Robustness of RFRS Algorithm

In the prior sections, we demonstrated the efficacy and the stealthiness of different attack types that leverage RFRS algorithm. We also demonstrated how different levels of steadiness impact the configurability and effectiveness of the attacks. We next demonstrate that if the scheme to pick the frames to attack is steadiness-agnostic, it will be less effective, less configurable, and less stealthy.

To evaluate this, we use three different steadiness-agnostic periodic frame-picking schemes. These schemes include (1) *FixedLength*, where the attacker launches the attack periodically after every N frames for a fixed attack length, e.g. 10 frames, (2) *EqualSum*, this schemes launches the attacks periodically but makes sure that the number of attacked frames is same as ours RFRS algorithm. (3) XPercent: in this scheme X% frames of the sequence are attacked.

Figure 12 demonstrates the use of periodic frame-picking schemes to launch a *slowdown* attack. Figure 12a shows that, using FixedLength frame-picking scheme, the attacker is not able to launch a consistent *slowdown* attack, as was the case for RFRS algorithm (Figure 8b). Furthermore, the total attacked frames are not equal to RFRS algorithm with attack length of 10. For a fair comparison, we used the EqualSum scheme where the total number of attacked frames is as RFRS. Figure 12b shows that the deviation increases, but the attacker is still not able to launch a precise attack.

The final configuration of the attack is shown in Figure 12c. While this attack seems to be highly effective, the graph does not paint a complete picture. Table 2 presents a comparison of all the attacks we have launched so far based on various metrics. We can see that the *ks* test value and the mean difference for the XPercent attack is the highest, which means that this attack is the least stealthy attack in our set. This will result in the attack being detected by the system, as it has changed the distribution of the similarity index for the sequence significantly after the attack.

Attack	Attack percentage	ks test statistic	Mean difference
SlowDown	7.2	0.06	0.14
Speedup	7.2	0.07	0.11
Zero Displacement	7.2	0.05	0.03
Path Deviation	7.2	0.07	0.15
FixedLength	4.5	0.07	0.23
EqualSum	7.2	0.08	0.35
XPercent	10	0.13	0.49

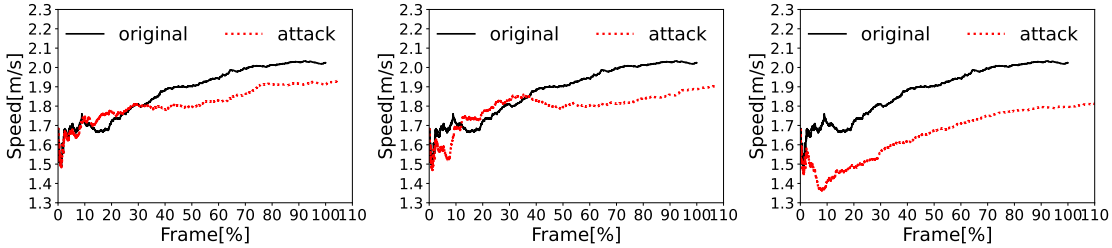
Table 2: ***Comparison of different frame picking schemes***

5.6 Dynamic Adaptation of Attacks

In prior evaluations, we selected a given attack, such as *slowdown*, and persisted with the attack for the entire duration. However, under a fast-changing environment or user behavior, the attacker may want to adapt the attack based on the scene or user's action. This necessitates that the attacker should be able to switch between the attacks at runtime, without sacrificing stealthiness, control, and preciseness of the attacks. Since the underlying attack mechanisms make changes on per-frame-basis, and do not assume anything about the future, different attack mechanisms can be selected for each frame. Furthermore, the RFRS algorithm allows the attacker to configure the length of history to make steadiness decisions, which enables the attacker to select between a first and slow transition between attacks. A smaller history window would gradually adapt to the new scenario while a larger history window would slowly adapt to ensure a graceful increase in the number of attacks launched.

Figure 13 illustrates different scenarios and the attacks we launch under each scenario. In the first segment of the sequence, the application is helping the user jog at a fixed speed and the attacker wants to increase the jogging speed of the user. To achieve that, she launches a *slowdown* attack to give the application an impression that the user is walking slowly and the application signals the user to increase her speed, achieving the attacker's goal. In the second segment, the user wants to walk in a straight line while reducing her speed to rest her body and the attacker wants to deviate the user from the straight line. To do that, the attacker launches a *zero displacement* attack, which results in the fluctuations in the user's speed while also deviating her from the straight path.

In the third segment, the user is again trying to jog at a fixed speed and the attacker wants to slow her down. To achieve that, the attacker launches a *speedup* attack giving the tracking application an impression that the user is speeding up. The feedback loop of the application then instructs the user to reduce her speed, which achieves the attacker's goal. In the final segment of the sequence, the user is trying to increase her speed and finish at her desired end point. However, the attacker decides to launch the *path deviation* attack and deviate the user from her destination. As we see in the figure, the application perceives that the user is maintaining a fixed speed and instructs her to jog faster. The increase in speed,



(a) EqualLength (attack length 10) (b) EqualSum (c) XPercent (10%)

Figure 12: *Difference between the speed on attacked sequence and original sequence.*

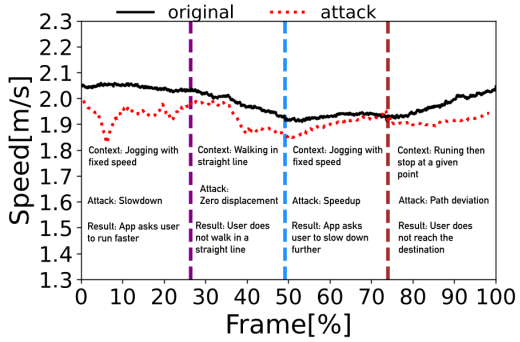


Figure 13: *Our attacks rely only on local information and allow attackers to change attacks based on the context.*

combined with the shift in deviation, results in the user losing the destination.

5.7 Microbenchmarks

We also present benchmarking results to demonstrate that our attacks are fast and simple, which allows them to be used in a fast-paced environment of MR. For a successful and stealthy launch, the attacker needs to decide on launching the attack, select an attack type, compute similarity index, and perform frame manipulations. In our use case, the visual stream has 5-30 fps rate, while inertial sensors have 12-20 fps. As a result, our attack should complete in less than 33.33ms, based on the attack vector with the highest sampling rate. We empirically calculated the time it takes to complete all of the different attack steps. The computation time of similarity index for the inertial vector on average is 0.8ms (1.5ms max and 0.7ms min). For visual attack vector, average computation time is 10ms (13.8ms max and 9.2ms min). The frame manipulation step takes 5ms on average. Finally, the RFRS algorithm runs in 0.0011 ms. The total time is less than 24ms on average, which is less than the highest sampling rate making our attacks fast and stealthy.

6 RELATED WORK

In this section, we discuss, and differentiate ourselves from, the related work in the domains of Mixed Reality (MR) security, spatiotemporal sensor attacks, and sensor fusion algorithms.

Mixed Reality (MR) Security. Prior research on MR security focuses on vulnerability analysis in applications interactions [5, 19, 20] secure authentication and pairing [10, 29, 32], security across-users and devices [26, 44], and privacy concerns in remote collaborations [15, 26]. No prior work explores the multi-modal and multi-dimensional attack surface presented here.

Spatiotemporal Sensor Attacks. There is a lot of prior work on sensor attacks across multiple dimensions. `Data-level` temporal and spatial attacks have been demonstrated for the inertial sensors [33, 34] as well as visual sensors [5, 36] using a range of manipulation strategies. `Device-level` manipulation of timing and location services has been shown to be effective as well [3, 4]. Prior work has also demonstrated spatiotemporal attacks that lead to misalignment of content in MR and inaccurate tracking [29]. However, these approaches ignore the concurrent launch of attacks across multiple attack vectors that is required to overcome the sensor fusion algorithms and systems checks for stealthy operation.

Sensor Fusion. The state-of-the-art sensor fusion algorithms leverage deep-learning based tracking on top of visual inertial odometry (VIO), as used by Hololens 2 [1]. Chen et al. [7] present robust deep learning-based multi-modal fusion for VIO by relying on the latent features from different sensor modalities, particularly under degraded environmental conditions such as noise, bias, occlusions, spatiotemporal misalignment, and missing data. In contrast, a malicious spatial and temporal misalignment would distort the fundamental characteristics of latent features, thus presenting new challenges in secure sensor fusion, and motivates further exploration on spatiotemporal attacks which is the focus of our work.

7 DISCUSSION

In this section, we discuss possible remediation techniques for our proposed attack surface and how most of these techniques will be ineffective in detecting and mitigating the proposed attacks. Additionally, we discuss how the proposed attacks are not only applicable to Mixed Reality (MR) but also apply to other relevant domains with similar spatiotemporal semantics.

Potential Defense Mechanisms. For the inertial sensors, prior work has proposed data production level defenses such as acoustic dampening [6] that help reduce resonant interference in the inertial signal. Others have proposed better filtering approaches to mitigate spoofing of inertial sensors [33]. For the visual stream, visual challenges can be an effective tool to detect camera attacks [36] and watermarks can be added to images to protect their integrity. However, these preventive strategies require either a fundamental shift in sensor hardware design or heavily instrumenting the environment, both are not feasible for practical and large-scale deployments. Also, our work shows that deep neural networks are ineffective in neutralizing concurrent manipulations across sensors and start adapting to malicious inputs induced by the attacker.

Other techniques include recovery methods like semantic analysis for capturing context information to evade the attacks. However, these techniques need a huge amount of contextual data labels along

with pose data that could be used to capture the abnormal data using statistical characteristics with semantic and context information [30]. Data authentication using cryptography [10] mechanism at sensor fusion level is resource-intensive for edge devices. Finally, online intrusion detection [13] using cryptography mechanisms like entropy, frame intervals, frame consistency and correlation, data fingerprinting can be used to detect attacked data frames. The solution presented by these techniques is to drop the attack frames at the tracking algorithm. If an attacker learns this behavior, the attacker might use this behavior against the tracking algorithm to cause large frame drops to induce further attacks.

Broad Applicability. Our approach has broader impacts both at different tracking scales and across different domains. For instance, our tracking use cases in navigation and exercising have movements that stretch from a few inches to meters. Our attack is also applicable to surgical use cases with micro-level movements. Also, our attack surface is transferable to other tracking-based systems such as autonomous vehicles [38], smart robots [42], and drones [9].

8 CONCLUSION

In this paper, we presented a novel multi-modal spatiotemporal attack surface for Mixed Reality (MR) systems. Our proposed attacks concurrently launch attacks on multiple sensing streams to achieve *fast*, *stealthy*, and *precise* outcomes. We proposed a generic framework to launch attacks that, in the case of a tracking application, can cause a user to slow down, speed up, deviate from the intended path, and lose the destination. We evaluated the efficacy of our proposed attacks against the state-of-the-art sensor fusion-based tracking algorithm and demonstrated that the attacker is able to achieve the goal across a wide range of environments and user action scenarios. In future we plan to explore machine learning techniques for finding the right time to attack, and context-aware optimization techniques for maximizing the attack impact and to keep it stealthy. We also aim to devise sensor fusion based defense mechanism to mitigate the propose attack surface.

REFERENCES

[1] 2020. Hololens 2. https://www.microsoft.com/en-us/hololens/.
[2] Sean Andrist, Dan Bohus, Ashley Feniello, and Nick Saw. 2022. Developing Mixed Reality Applications with Platform for Situated Intelligence. *IEEE VRW*.
[3] F.M. Anwar, L. Garcia, X. Han, and M. Srivastava. 2019. Securing Time in Untrusted Operating Systems with TimeSeal. *IEEE RTSS*.
[4] F.M. Anwar and M. Srivastava. 2020. A Case for Feedforward Control with Feedback Trim to Mitigate Time Transfer Attacks. *ACM TOPS*.
[5] P. Casey, I. Baggili, and A. Yarramreddy. 2019. Immersive Virtual Reality Attacks and the Human Joystick. *IEEE TDSC*.
[6] Simon Castro, Robert Dean, Grant Roth, George T Flowers, and Brian Grantham. 2007. Influence of Acoustic Noise on the Dynamic Performance of MEMS Gyroscopes. *ASME IMECE*.
[7] Changhao Chen, Stefano Rosa, Yishu Miao, Chris Xiaoxuan Lu, Wei Wu, Andrew Markham, and Niki Trigoni. 2019. Selective sensor fusion for neural visual-inertial odometry. *IEEE/CVF CVPR*.
[8] Ronald Clark, Sen Wang, Hongkai Wen, A. Markham, and Agathoniki Trigoni. 2017. VINet: Visual-Inertial Odometry as a Sequence-to-Sequence Learning Problem. *AAAI*.
[9] Louis Dressel and Mykel J. Kochenderfer. 2019. Hunting Drones with Other Drones: Tracking a Moving Radio Target. *ICRA*.
[10] Ethan Gaebel, Ning Zhang, Wenjing Lou, and Y. Thomas Hou. 2016. Looks Good To Me: Authentication for Augmented Reality. *ACM TrustED*.
[11] Cassandra Gorman and Louise Gustafsson. 2022. The use of augmented reality for rehabilitation after stroke: a narrative review. *DRAT*.
[12] Thomas M Gregory, Jules Gregory, John Sledge, Romain Allard, and Olivier Mir. 2018. Surgery guided by mixed reality: presentation of a proof of concept. *Acta orthopaedica*.
[13] Bogdan Groza and Pal-Stefan Murvay. 2019. Efficient Intrusion Detection With Bloom Filtering in Controller Area Networks. *IEEE TIFS*.
[14] Qingying Hao, Licheng Luo, Steve T.K. Jan, and Gang Wang. 2021. It's Not What It Looks Like: Manipulating Perceptual Hashing Based Applications. *ACM CCS*.
[15] Jassim Happa, Mashhuda Glencross, and Anthony Steed. 2019. Cyber Security Threats and Challenges in Collaborative Mixed-Reality. *Frontiers in ICT*.
[16] Zhitao He and Thiemo Voigt. 2013. Droplet: A New Denial-of-Service Attack on Low Power Wireless Sensor Networks. *IEEE MASS*.
[17] Nicolas Kakouris, Numan Yener, and Daniel T.P. Fong. 2021. A systematic review of running-related musculoskeletal injuries in runners. *Journal of SHS*.
[18] Anunay Kulshrestha. 2012. On the Hamming Distance between base-n representations of whole numbers. *ArXiv*.
[19] Kiron Lebeck, Tadayoshi Kohno, and Franziska Roesner. 2016. How to Safely Augment Reality: Challenges and Directions. *ACM HotMobile*.
[20] Kiron Lebeck, Tadayoshi Kohno, and Franziska Roesner. 2019. Enabling Multiple Applications to Simultaneously Augment Reality: Challenges and Directions. *ACM HotMobile*.
[21] Mingyang Li and Anastasios Mourikis. 2013. High-precision, consistent EKF-based visual–inertial odometry. *Journal of Robotics Research*.
[22] Yonggen Ling, Linchao Bao, Zequn Jie, Fengming Zhu, Ziyang Li, Shanmin Tang, Yongsheng Liu, W. Liu, and T. Zhang. 2018. Modeling Varying Camera-IMU Time Offset in Optimization-Based Visual-Inertial Odometry. *ECCV*.
[23] Fangfang Lu, Hao Zhou, Lingling Guo, Jingjing Chen, and Licheng Pei. 2021. An ARCore-Based Augmented Reality Campus Navigation System. *Applied Sciences*.
[24] Frank J Massey Jr. 1951. The Kolmogorov-Smirnov test for goodness of fit. *ASA*.
[25] Deeraj Nagothu, Yu Chen, Erik Blasch, Alexander Aved, and Sencun Zhu. 2019. Detecting Malicious False Frame Injection Attacks on Surveillance Systems at the Edge Using Electrical Network Frequency Signals. *Sensors*.
[26] Xukan Ran, Carter Slocum, Maria Gorlatova, and Jiasi Chen. 2019. ShareAR: Communication-Efficient Multi-User Mobile Augmented Reality. *ACM HotNets*.
[27] Priyanka Samanta and Shweta Jain. 2021. Analysis of Perceptual Hashing Algorithms in Image Manipulation Detection. *Procedia CS*.
[28] Thomas Schöps, Torsten Sattler, and Marc Pollefeys. 2019. BAD SLAM: Bundle Adjusted Direct RGB-D SLAM. *IEEE/CVF CVPR*.
[29] R. A. Sharma, A. Dongare, J. Miller, N. Wilkerson, D. Cohen, V. Sekar, P. Dutta, and A. Rowe. 2020. All that GLITTERs: Low-Power Spoof-Resilient Optical Markers for Augmented Reality. *ACM IPSN*.
[30] Amit Kumar Sikder, Hidayet Aksu, and A. Selcuk Uluagac. 2017. 6thSense: A Context-aware Sensor-based Attack Detector for Smart Devices. *USENIX Security*.
[31] Vivek Singh, Pallav Pant, and Ramesh Tripathi. 2015. Detection of Frame Duplication Type of Forgery in Digital Video Using Sub-block Based Features. *ICDF2C*.
[32] I. Sluganovic, M. Serbec, A. Derek, and I. Martinovic. 2017. HoloPair: Securing Shared Augmented Reality Using Microsoft HoloLens. *ACSAC*.
[33] Timothy Trippel, Ofir Weisse, Wenyuan Xu, Peter Honeyman, and Kevin Fu. 2017. WALNUT: Waging Doubt on the Integrity of MEMS Accelerometers with Acoustic Injection Attacks. *IEEE EuroS&P*.
[34] Y. Tu, Z. Lin, I. Lee, and X. Hei. 2018. Injected and Delivered: Fabricating Implicit Control over Actuation Systems by Spoofing Inertial Sensors. *USENIX Security*.
[35] Dorin Ungureanu, Federica Bogo, S. Galliani, Pooja Sama, Xin Duan, Casey Meekhof, Jan Stühmer, Thomas J. Cashman, Bugra Tekin, Johannes L. Schönberger, Pawel Olszta, and Marc Pollefeys. 2020. HoloLens 2 Research Mode as a Tool for Computer Vision Research. *ArXiv*.
[36] J. Valente, K. Bahirat, K. Venechanos, A.A. Cardenas, and P. Balakrishnan. 2019. Improving the Security of Visual Challenges. *ACM TCPS*.
[37] Mathy Vanhoef. 2021. Fragment and Forge: Breaking Wi-Fi Through Frame Aggregation and Fragmentation. *USENIX Security*.
[38] Heng Wang and Xiaodong Zhang. 2022. Real-time vehicle detection and tracking using 3D LiDAR. *Asian Journal of Control*.
[39] Chris Warin and Delphine Reinhardt. 2022. Vision: Usable Privacy for XR in the Era of the Metaverse. *EuroUSEC*.
[40] Carolin Wienrich, Philipp Komma, Stephanie Vogt, and Marc E. Latoschik. 2021. Spatial Presence in Mixed Realities–Considerations About the Concept, Measures, Design, and Experiments. *Frontiers in VR*.
[41] Yi Xu, Yuzhang Wu, and Hui Zhou. 2018. Multi-Scale Voxel Hashing and Efficient 3D Representation for Mobile Augmented Reality. *IEEE CVPR Workshops*.
[42] Hui Zhang, Li Zhu Liu, He Xie, Yiming Jiang, Jian Zhou, and Yaonan Wang. 2022. Deep Learning-Based Robot Vision: High-End Tools for Smart Manufacturing. *IEEE IMM*.
[43] Xiaoming Zhao, Jingmeng Liu, Xingming Wu, Weihai Chen, Fanghong Guo, and Zhengguo Li. 2022. Probabilistic Spatial Distribution Prior Based Attentional Keypoints Matching Network. *IEEE TCSVT*.
[44] Fengyuan Zhu and Tovi Grossman. 2020. BISHARE: Exploring Bidirectional Interactions Between Smartphones and Head-Mounted Augmented Reality. *ACM CHI*.